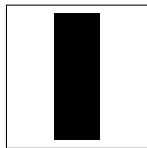


5

Dictionary Techniques

5.1 Overview



In the previous two chapters we looked at coding techniques that assume a source that generates a sequence of independent symbols. As most sources are correlated to start with, the coding step is generally preceded by a decorrelation step. In this chapter we will look at techniques that incorporate the structure in the data in order to increase the amount of compression. These techniques—both static and adaptive (or dynamic)—build a list of commonly occurring patterns and encode these patterns by transmitting their index in the list. They are most useful with sources that generate a relatively small number of patterns quite frequently, such as text sources and computer commands. We discuss applications to text compression, modem communications, and image compression.

5.2 Introduction

In many applications, the output of the source consists of recurring patterns. A classic example is a text source in which certain patterns or words recur constantly. Also, there are certain patterns that simply do not occur, or if they do, occur with great rarity. For example, we can be reasonably sure that the word *Limpopo*¹ occurs in a very small fraction of the text sources in existence.

A very reasonable approach to encoding such sources is to keep a list, or *dictionary*, of frequently occurring patterns. When these patterns appear in the source output, they are encoded with a reference to the dictionary. If the pattern does not appear in the dictionary, then it can be encoded using some other, less efficient, method. In effect we are splitting

¹ “How the Elephant Got Its Trunk” in *Just So Stories* by Rudyard Kipling.

the input into two classes, frequently occurring patterns and infrequently occurring patterns. For this technique to be effective, the class of frequently occurring patterns, and hence the size of the dictionary, must be much smaller than the number of all possible patterns.

Suppose we have a particular text that consists of four-character words, three characters from the 26 lowercase letters of the English alphabet followed by a punctuation mark. Suppose our source alphabet consists of the 26 lowercase letters of the English alphabet and the punctuation marks comma, period, exclamation mark, question mark, semicolon, and colon. In other words, the size of the input alphabet is 32. If we were to encode the text source one character at a time, treating each character as an equally likely event, we would need 5 bits per character. Treating all 32^4 ($= 2^{20} = 1,048,576$) four-character patterns as equally likely, we have a code that assigns 20 bits to each four-character pattern. Let us now put the 256 most likely four-character patterns into a dictionary. The transmission scheme works as follows: Whenever we want to send a pattern that exists in the dictionary, we will send a 1-bit flag, say, a 0, followed by an 8-bit index corresponding to the entry in the dictionary. If the pattern is not in the dictionary, we will send a 1 followed by the 20-bit encoding of the pattern. If the pattern we encounter is not in the dictionary, we will actually use more bits than in the original scheme, 21 instead of 20. But if it is in the dictionary, we will send only 9 bits. The utility of our scheme will depend on the percentage of the words we encounter that are in the dictionary. We can get an idea about the utility of our scheme by calculating the average number of bits per pattern. If the probability of encountering a pattern from the dictionary is p , then the average number of bits per pattern R is given by

$$R = 9p + 21(1 - p) = 21 - 12p. \quad (5.1)$$

For our scheme to be useful, R should have a value less than 20. This happens when $p \geq 0.084$. This does not seem like a very large number. However, note that if all patterns were occurring in an equally likely manner, the probability of encountering a pattern from the dictionary would be less than 0.00025!

We do not simply want a coding scheme that performs slightly better than the simple-minded approach of coding each pattern as equally likely; we would like to improve the performance as much as possible. In order for this to happen, p should be as large as possible. This means that we should carefully select patterns that are most likely to occur as entries in the dictionary. To do this, we have to have a pretty good idea about the structure of the source output. If we do not have information of this sort available to us prior to the encoding of a particular source output, we need to acquire this information somehow when we are encoding. If we feel we have sufficient prior knowledge, we can use a *static* approach; if not, we can take an *adaptive* approach. We will look at both these approaches in this chapter.

5.3 Static Dictionary

Choosing a static dictionary technique is most appropriate when considerable prior knowledge about the source is available. This technique is especially suitable for use in specific applications. For example, if the task were to compress the student records at a university, a static dictionary approach may be the best. This is because we know ahead of time that certain words such as “Name” and “Student ID” are going to appear in almost all of the records.

Other words such as “Sophomore,” “credits,” and so on will occur quite often. Depending on the location of the university, certain digits in social security numbers are more likely to occur. For example, in Nebraska most student ID numbers begin with the digits 505. In fact, most entries will be of a recurring nature. In this situation, it is highly efficient to design a compression scheme based on a static dictionary containing the recurring patterns. Similarly, there could be a number of other situations in which an application-specific or data-specific static-dictionary-based coding scheme would be the most efficient. It should be noted that these schemes would work well only for the applications and data they were designed for. If these schemes were to be used with different applications, they may cause an expansion of the data instead of compression.

A static dictionary technique that is less specific to a single application is *digram coding*. We describe this in the next section.

5.3.1 Digram Coding

One of the more common forms of static dictionary coding is digram coding. In this form of coding, the dictionary consists of all letters of the source alphabet followed by as many pairs of letters, called *digrams*, as can be accommodated by the dictionary. For example, suppose we were to construct a dictionary of size 256 for digram coding of all printable ASCII characters. The first 95 entries of the dictionary would be the 95 printable ASCII characters. The remaining 161 entries would be the most frequently used pairs of characters.

The digram encoder reads a two-character input and searches the dictionary to see if this input exists in the dictionary. If it does, the corresponding index is encoded and transmitted. If it does not, the first character of the pair is encoded. The second character in the pair then becomes the first character of the next digram. The encoder reads another character to complete the digram, and the search procedure is repeated.

Example 5.3.1:

Suppose we have a source with a five-letter alphabet $\mathcal{A} = \{a, b, c, d, r\}$. Based on knowledge about the source, we build the dictionary shown in Table 5.1.

TABLE 5.1 A sample dictionary.

Code	Entry	Code	Entry
000	<i>a</i>	100	<i>r</i>
001	<i>b</i>	101	<i>ab</i>
010	<i>c</i>	110	<i>ac</i>
011	<i>d</i>	111	<i>ad</i>

Suppose we wish to encode the sequence

abracadabra

The encoder reads the first two characters *ab* and checks to see if this pair of letters exists in the dictionary. It does and is encoded using the codeword 101. The encoder then reads

the next two characters *ra* and checks to see if this pair occurs in the dictionary. It does not, so the encoder sends out the code for *r*, which is 100, then reads in one more character, *c*, to make the two-character pattern *ac*. This does exist in the dictionary and is encoded as 110. Continuing in this fashion, the remainder of the sequence is coded. The output string for the given input sequence is 10110011011101100000. ♦

TABLE 5.2 Thirty most frequently occurring pairs of characters in a 41,364-character-long LaTeX document.

Pair	Count	Pair	Count
<i>eb</i>	1128	<i>ar</i>	314
<i>bt</i>	838	<i>at</i>	313
<i>bb</i>	823	<i>bw</i>	309
<i>th</i>	817	<i>te</i>	296
<i>he</i>	712	<i>bs</i>	295
<i>in</i>	512	<i>db</i>	272
<i>sb</i>	494	<i>bo</i>	266
<i>er</i>	433	<i>io</i>	257
<i>ba</i>	425	<i>co</i>	256
<i>tb</i>	401	<i>re</i>	247
<i>en</i>	392	<i>B\$</i>	246
<i>on</i>	385	<i>rb</i>	239
<i>nb</i>	353	<i>di</i>	230
<i>ti</i>	322	<i>ic</i>	229
<i>bi</i>	317	<i>ct</i>	226

TABLE 5.3 Thirty most frequently occurring pairs of characters in a collection of C programs containing 64,983 characters.

Pair	Count	Pair	Count
<i>bb</i>	5728	<i>st</i>	442
<i>nlb</i>	1471	<i>le</i>	440
<i>;nl</i>	1133	<i>ut</i>	440
<i>in</i>	985	<i>f(</i>	416
<i>nt</i>	739	<i>ar</i>	381
<i>=b</i>	687	<i>or</i>	374
<i>bi</i>	662	<i>rb</i>	373
<i>tb</i>	615	<i>en</i>	371
<i>b=</i>	612	<i>er</i>	358
<i>);</i>	558	<i>ri</i>	357
<i>,b</i>	554	<i>at</i>	352
<i>nl nl</i>	506	<i>pr</i>	351
<i>bf</i>	505	<i>te</i>	349
<i>eb</i>	500	<i>an</i>	348
<i>b*</i>	444	<i>lo</i>	347

A list of the 30 most frequently occurring pairs of characters in an earlier version of this chapter is shown in Table 5.2. For comparison, the 30 most frequently occurring pairs of characters in a set of C programs is shown in Table 5.3.

In these tables, *␣* corresponds to a space and *nl* corresponds to a new line. Notice how different the two tables are. It is easy to see that a dictionary designed for compressing L^AT_EX documents would not work very well when compressing C programs. However, generally we want techniques that will be able to compress a variety of source outputs. If we wanted to compress computer files, we do not want to change techniques based on the content of the file. Rather, we would like the technique to *adapt* to the characteristics of the source output. We discuss adaptive-dictionary-based techniques in the next section.

5.4 Adaptive Dictionary

Most adaptive-dictionary-based techniques have their roots in two landmark papers by Jacob Ziv and Abraham Lempel in 1977 [54] and 1978 [55]. These papers provide two different approaches to adaptively building dictionaries, and each approach has given rise to a number of variations. The approaches based on the 1977 paper are said to belong to the LZ77 family (also known as LZ1), while the approaches based on the 1978 paper are said to belong to the LZ78, or LZ2, family. The transposition of the initials is a historical accident and is a convention we will observe in this book. In the following sections, we first describe an implementation of each approach followed by some of the more well-known variations.

5.4.1 The LZ77 Approach

In the LZ77 approach, the dictionary is simply a portion of the previously encoded sequence. The encoder examines the input sequence through a sliding window as shown in Figure 5.1. The window consists of two parts, a *search buffer* that contains a portion of the recently encoded sequence, and a *look-ahead buffer* that contains the next portion of the sequence to be encoded. In Figure 5.1, the search buffer contains eight symbols, while the look-ahead buffer contains seven symbols. In practice, the sizes of the buffers are significantly larger; however, for the purpose of explanation, we will keep the buffer sizes small.

To encode the sequence in the look-ahead buffer, the encoder moves a search pointer back through the search buffer until it encounters a match to the first symbol in the look-ahead

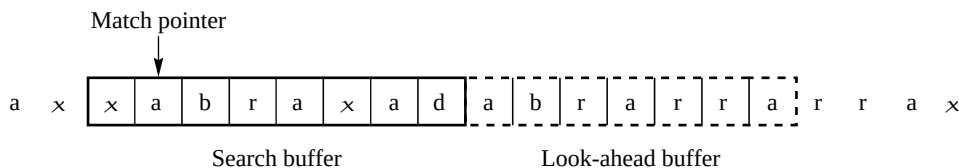


FIGURE 5.1 Encoding using the LZ77 approach.

buffer. The distance of the pointer from the look-ahead buffer is called the *offset*. The encoder then examines the symbols following the symbol at the pointer location to see if they match consecutive symbols in the look-ahead buffer. The number of consecutive symbols in the search buffer that match consecutive symbols in the look-ahead buffer, starting with the first symbol, is called the length of the match. The encoder searches the search buffer for the longest match. Once the longest match has been found, the encoder encodes it with a triple $\langle o, l, c \rangle$, where o is the offset, l is the length of the match, and c is the codeword corresponding to the symbol in the look-ahead buffer that follows the match. For example, in Figure 5.1 the pointer is pointing to the beginning of the longest match. The offset o in this case is 7, the length of the match l is 4, and the symbol in the look-ahead buffer following the match is ρ .

The reason for sending the third element in the triple is to take care of the situation where no match for the symbol in the look-ahead buffer can be found in the search buffer. In this case, the offset and match-length values are set to 0, and the third element of the triple is the code for the symbol itself.

If the size of the search buffer is S , the size of the window (search and look-ahead buffers) is W , and the size of the source alphabet is A , then the number of bits needed to code the triple using fixed-length codes is $\lceil \log_2 S \rceil + \lceil \log_2 W \rceil + \lceil \log_2 A \rceil$. Notice that the second term is $\lceil \log_2 W \rceil$, not $\lceil \log_2 S \rceil$. The reason for this is that the length of the match can actually exceed the length of the search buffer. We will see how this happens in Example 5.4.1.

In the following example, we will look at three different possibilities that may be encountered during the coding process:

1. There is no match for the next character to be encoded in the window.
2. There is a match.
3. The matched string extends inside the look-ahead buffer.

Example 5.4.1: The LZ77 approach

Suppose the sequence to be encoded is

...cabracadabrarrrrad...

Suppose the length of the window is 13, the size of the look-ahead buffer is six, and the current condition is as follows:

cabraca	dabrar
---------	--------

with *dabrar* in the look-ahead buffer. We look back in the already encoded portion of the window to find a match for *d*. As we can see, there is no match, so we transmit the triple $\langle 0, 0, C(d) \rangle$. The first two elements of the triple show that there is no match to *d* in the search buffer, while $C(d)$ is the code for the character *d*. This seems like a wasteful way to encode a single character, and we will have more to say about this later.

For now, let's continue with the encoding process. As we have encoded a single character, we move the window by one character. Now the contents of the buffer are

abracad	abrarr
---------	--------

with *abrarr* in the look-ahead buffer. Looking back from the current location, we find a match to *a* at an offset of two. The length of this match is one. Looking further back, we have another match for *a* at an offset of four; again the length of the match is one. Looking back even further in the window, we have a third match for *a* at an offset of seven. However, this time the length of the match is four (see Figure 5.2). So we encode the string *abra* with the triple $\langle 7, 4, C(r) \rangle$, and move the window forward by five characters. The window now contains the following characters:

adabrar	rarrad
---------	--------

Now the look-ahead buffer contains the string *rarrad*. Looking back in the window, we find a match for *r* at an offset of one and a match length of one, and a second match at an offset of three with a match length of what at first appears to be three. It turns out we can use a match length of five instead of three.

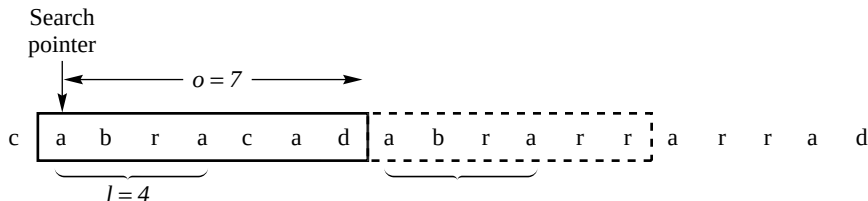


FIGURE 5.2 The encoding process.

Why this is so will become clearer when we decode the sequence. To see how the decoding works, let us assume that we have decoded the sequence *cabraca* and we receive the triples $\langle 0, 0, C(d) \rangle$, $\langle 7, 4, C(r) \rangle$, and $\langle 3, 5, C(d) \rangle$. The first triple is easy to decode; there was no match within the previously decoded string, and the next symbol is *d*. The decoded string is now *cabracad*. The first element of the next triple tells the decoder to move the copy pointer back seven characters, and copy four characters from that point. The decoding process works as shown in Figure 5.3.

Finally, let's see how the triple $\langle 3, 5, C(d) \rangle$ gets decoded. We move back three characters and start copying. The first three characters we copy are *rar*. The copy pointer moves once again, as shown in Figure 5.4, to copy the recently copied character *r*. Similarly, we copy the next character *a*. Even though we started copying only three characters back, we end up decoding five characters. Notice that the match only has to *start* in the search buffer; it can extend into the look-ahead buffer. In fact, if the last character in the look-ahead buffer

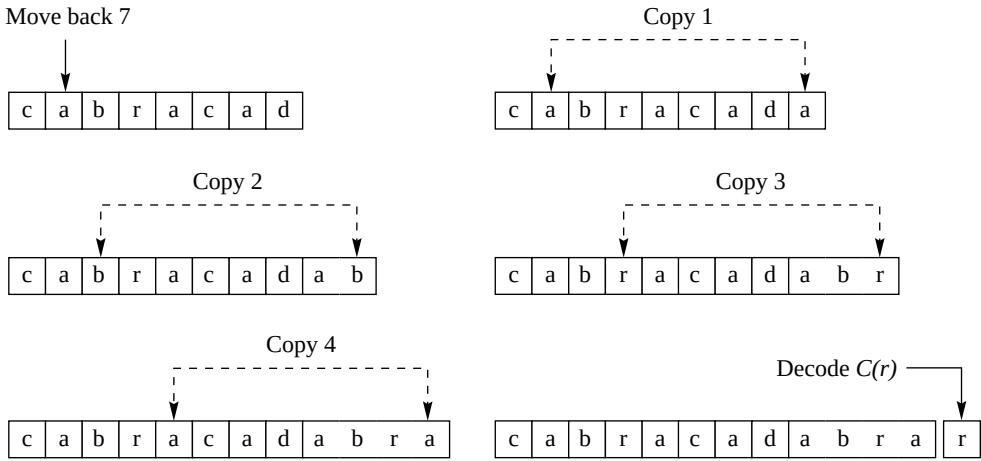


FIGURE 5.3 Decoding of the triple $\langle 7, 4, C(r) \rangle$.

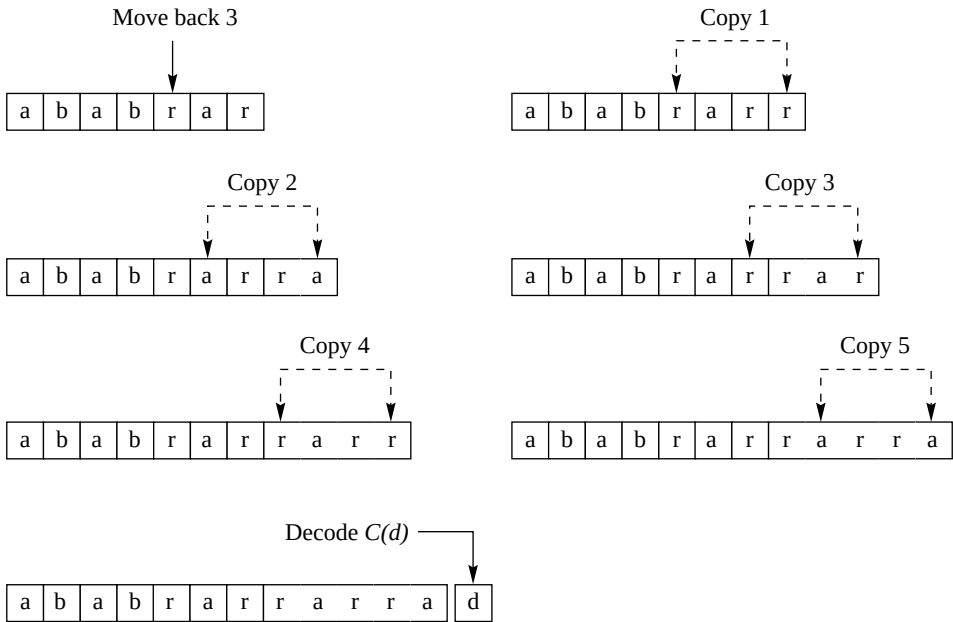


FIGURE 5.4 Decoding the triple $\langle 3, 5, C(d) \rangle$.

had been r instead of d , followed by several more repetitions of rar , the entire sequence of repeated $rars$ could have been encoded with a single triple. ♦

As we can see, the LZ77 scheme is a very simple adaptive scheme that requires no prior knowledge of the source and seems to require no assumptions about the characteristics of the source. The authors of this algorithm showed that asymptotically the performance of this algorithm approached the best that could be obtained by using a scheme that had full knowledge about the statistics of the source. While this may be true asymptotically, in practice there are a number of ways of improving the performance of the LZ77 algorithm as described here. Furthermore, by using the recent portions of the sequence, there is an assumption of sorts being used here—that is, that patterns recur “close” together. As we shall see, in LZ78 the authors removed this “assumption” and came up with an entirely different adaptive-dictionary-based scheme. Before we get to that, let us look at the different variations of the LZ77 algorithm.

Variations on the LZ77 Theme

There are a number of ways that the LZ77 scheme can be made more efficient, and most of these have appeared in the literature. Many of the improvements deal with the efficient encoding of the triples. In the description of the LZ77 algorithm, we assumed that the triples were encoded using a fixed-length code. However, if we were willing to accept more complexity, we could encode the triples using variable-length codes. As we saw in earlier chapters, these codes can be adaptive or, if we were willing to use a two-pass algorithm, they can be semiadaptive. Popular compression packages, such as PKZip, Zip, LHarc, PNG, gzip, and ARJ, all use an LZ77-based algorithm followed by a variable-length coder.

Other variations on the LZ77 algorithm include varying the size of the search and look-ahead buffers. To make the search buffer large requires the development of more effective search strategies. Such strategies can be implemented more effectively if the contents of the search buffer are stored in a manner conducive to fast searches.

The simplest modification to the LZ77 algorithm, and one that is used by most variations of the LZ77 algorithm, is to eliminate the situation where we use a triple to encode a single character. Use of a triple is highly inefficient, especially if a large number of characters occur infrequently. The modification to get rid of this inefficiency is simply the addition of a flag bit, to indicate whether what follows is the codeword for a single symbol. By using this flag bit we also get rid of the necessity for the third element of the triple. Now all we need to do is to send a pair of values corresponding to the offset and length of match. This modification to the LZ77 algorithm is referred to as LZSS [56, 57].

5.4.2 The LZ78 Approach

The LZ77 approach implicitly assumes that like patterns will occur close together. It makes use of this structure by using the recent past of the sequence as the dictionary for encoding.